

Spring Ai 是基于大语言模型 LLM (文心一言, 通义千问, chatGPT, 豆包, Claude 等)的框架, 用于开发对应的 AI 应用程序
是 java spring 框架, 类似 java 的 LangChain4J, python 的 LangChain

LLM 使用到的技术:

Generative -- 生成式, 根据上文预测应该出现的文本或数据, 是连续输出的
(所以大模型的结果不是一下就出来的, 而是逐步出来的)

Pre-trained -- 预训练, 通过大规模数据进行训练, 让大模型认识人类的语法

Transformer -- 一种处理自然语言的神经网络模型

比如输入: 今天吃啥, 他会得到很多字以及对应的概率

比如: 吃 80%, 推 10% 这种, 然后选吃

然后就是将 今天吃啥, 吃 放入 transformer, 得到下一个字, 从而最后得到结果

大模型应用的方案:

1.纯 Prompt 问答 -- 直接调用大模型, 基于它本身的推理能力进行问答

Prompt 就是调用大模型需要发送的提示词 / 内容

2.Agent + Function calling -- 大模型进行任务的拆解, 然后业务端调用对应的接口实现业务

3.RAG (Retrieval Augmented Generation) -- 给大模型外挂一个知识库, 让大模型基于知识库的内容进行问答 (对于知识库里面的内容, 我们先检索出相关的内容, 然后再把这个知识片段发过去, 不是真的让大模型直接查询这个知识库)

因为大模型有 知识的 领域限制 (只有通用的知识, 没有专业知识) / 时间限制 (知识不是最新的)

4.Fine-tuning -- 针对大模型进行微调, 来满足特定的需求

阿里云大模型的 api-key: sk-26450bac21234edc8c8b9a40c1581d93

提示词工程:

通过优化提示词, 使大模型生成尽可能理想的内容

1. 清晰明确的指令 -- 比如: 说一下人工智能 / 说一下人工智能的应用领域, 以及 3 个案例, 肯定是后者更准确

2. 使用分隔符标记输入 -- 比如: 将某个语句进行解释, 然后输入: "abc"进行解释,

3. 按步骤拆解复杂任务 -- 用户输入了一个任务, 然后我们传递给大模型的提示词, 可以先将任务拆分成多个步骤, 再将提示词传过去

比如用户输入: 计算 $a+2=b$, $b=3$, a 是多少

我们传递过去的是: 请用以下步骤处理用户的数学问题, 步骤 1: 计算答案, 并显示完整计算逻辑, 步骤 2: 验证答案是否正确, 用户输入: 计算 $a+2=b$, $b=3$, a 是多少

4. 提供输入输出示例 -- 在 systemMessage 里面 (defaultSystem), 加上一个输入和输出的示例, 输入就写进 user, 输出就写进 assistant, 这样大模型的答案会参考这个输入输出案例来生成

5. 明确要求输出格式 -- 比如: 解析用户输入, 以 json 格式输出, 包含 sql 和对应的解释, 以及案例数据

比如：解析用户输入，以 json 格式输出，包含：物品、客户、数量、要求

然后用户输入：帮我点 2 个拿铁，送到海康，写何先生收，要冰的

大模型输出：{"物品":"拿铁","客户":"何先生","数量": 2,"要求":"冰的"}

这样可以方便直接得到可以查询的条件

6. 给模型设定一个角色 -- 比如：你是一个数据库方面的专家，负责将用户输入转化成对应 sql

Agent + Function calling:

原始流程是：1. 给大模型设定一个角色以及功能，然后说明它的功能的实现流程

2. 每个需要调用接口获取数据的时候，都让大模型返回入参和调用函数

3. 我们获取到函数和入参之后，通过反射的方式调用这些方法，并将返回数据拼接，然后再次传给大模型，从而得到下一个需要做的事情，直到最终完成任务

比如：给用户推荐课程的流程

1. 问用户的兴趣爱好 -- 这一步是大模型来对接并得到

2. 根据爱好得到相关的课程 -- 这一步是大模型来判断需要调用的接口是哪个，并把入参传递过来，然后用反射的方式调用方法得到课程 (Spring Ai 封装了这一步，所以我们实际不需要写反射代码)

3. 将课程告知用户并引导用户留下联系方式并试听 -- 这一步也是大模型

4. 根据联系方式进行预约 -- 这一步就是大模型告知需要调用预约的接口，然后传递用户的信息，然后我们反射的去调用方法来预约

Spring Ai 流程：1. 编写提示词

2. 定义 Function (写一个工具类，通过 @Tool 和 @ToolParam 注解，让 SpringAi 知道哪些是 Function，从而可以传递给大模型，这也是 springAi 封装的反射代码)

3. 配置 Function (将我们的工具类，在创建 ChatClient 的时候，通过 defaultTools 参数传递进去，从而让大模型知道工具类是哪个)

RAG:

向量：空间中的两个点，从起点到这两个点的线就是向量

判断两个向量是否接近的算法 -- 欧式距离 -- 直接计算两点之间的距离 (越小越接近)

余弦距离 -- 计算两个线之间的夹角的 cos 值 (角度越小越接近，所以 cos 值越大越接近)

不管是文字、图片、视频等数据，都可以通过算法转化成一个个向量，从而可以计算相似值 (这样就可以通过含义是否相近来找数据，而不需要通过字是否一致来找 -- 哪怕是 ES 也只是模糊查询，不能语义查询)

向量模型：将数据向量化的模型

就是不断的训练，将语义接近的东西在向量中也接近

步骤：1. 引入对应的模型依赖 (看你想要实现什么功能，就有对应的好的依赖)
2. 配置模型 (在配置文件中配置模型，就是 `spring.ai.embedding` 参数)
3. 使用向量模型 (使用 `EmbeddingModel`)，将数据转化成向量 -- 可以看 `DemoAiApplicationTests` 类，在里面有个使用向量模型的案例 (因为实际是使用向量数据库，所以这个计算向量的过程不是我们实现的)

向量数据库：正常的向量模型需要一个个数据的计算来获取向量结果，这样每次比对都有大量的计算，而向量数据库就是在存入数据的时候就进行了向量计算，从而查询的时候直接得到结果

有很多向量数据库，有原生的，也有非原生的 (就是本来是其他数据库，最近加了向量数据库的功能)

原生的：Milvus, Qdrant 等

非原生的：ES, Oracle, Redis 等

步骤：1. 引入对应向量数据库的依赖 (不管什么数据库，springAi 都做了抽象，所以我们代码写的其实差不多)

2. 配置向量数据库

3. 读写数据

我这使用的是 `SimpleVectorStore`，是 springAi 模拟的向量数据库，所以不需要安装数据库，也不需要引入依赖和配置，可以直接读写，不过读写代码是和正常的向量数据库一样的 -- 可以看 `DemoAiApplicationTests` 类

RAG 原始流程：1. 上传文件/数据，将这些数据保存进向量数据库

2. 每次用户问问题的时候，将问题去向量数据库检索，得到最接近的知识片段

3. 拼接问题和知识片段，一起发送给大模型，从而得到答案

Spring Ai 流程：1. 编写提示词

2. 配置 RAG 的 `Advisor` (将我们的向量数据库，在创建 `ChatClient` 的时候，通过 `defaultAdvisors` 参数传递进去，从而让大模型知道向量数据库是哪个)